

ZADU: A Python Library for Evaluating the Reliability of Dimensionality Reduction Embeddings

Hyeon Jeon^{*1}, Aeri Cho^{*1}, Jinhwa Jang^{*1,3}, Soohyun Lee^{*1}, Jake Hyun^{§1},
Hyung-Kwon Ko^{†2}, Jaemin Jo^{‡4}, and Jinwook Seo^{§1}

¹Seoul National University ²KAIST ³Samsung Electronics ⁴Sungkyunkwan University

ABSTRACT

Dimensionality reduction (DR) techniques inherently distort the original structure of input high-dimensional data, producing imperfect low-dimensional embeddings. Diverse distortion measures have thus been proposed to evaluate the reliability of DR embeddings. However, implementing and executing distortion measures in practice has so far been time-consuming and tedious. To address this issue, we present ZADU, a Python library that provides distortion measures. ZADU is not only easy to install and execute but also enables comprehensive evaluation of DR embeddings through three key features. First, the library covers a wide range of distortion measures. Second, it automatically optimizes the execution of distortion measures, substantially reducing the running time required to execute multiple measures. Last, the library informs how individual points contribute to the overall distortions, facilitating the detailed analysis of DR embeddings. By simulating a real-world scenario of optimizing DR embeddings, we verify that our optimization scheme substantially reduces the time required to execute distortion measures. Finally, as an application of ZADU, we present another library called ZADUvis that allows users to easily create distortion visualizations that depict the extent to which each region of an embedding suffers from distortions.

Index Terms: Human-centered computing—Visualization—Visualization design and evaluation methods

1 INTRODUCTION

Dimensionality reduction (DR) suffers from inaccuracy. Although DR is a useful technique for visually analyzing high-dimensional data [32], distortion inevitably occurs while moving data from a broad high-dimensional space to a narrow low-dimensional space [16, 18, 28, 32]. Such distortions lower the credibility of data analysis with DR embeddings. To avoid such risks of misinterpretation, we need to assess the reliability of the embeddings prior to their usage. For this purpose, various distortion measures (e.g., Trustworthiness & Continuity [25] and Steadiness & Cohesiveness [18]) have been proposed [32].

However, there is a lack of an easy-to-use library that provides distortion measures, which leads to the consumption of researchers' valuable time. A few research works provide the source code of distortion measures [10, 15, 19] (Table 1). However, researchers need considerable time to install and execute such code. For example, they need to manually configure the environment settings and install the dependencies. Researchers thus often implement distortion measures on their own, but the laboriousness of the task persists.

^{*}e-mail: {hj, archio, jhjang, shlee}@hcil.snu.ac.kr

[†]hyungkwnko@gmail.com

[‡]jmjo@skku.edu

[§]{jakehyun, jseo}@snu.ac.kr

Given this background, we present ZADU, a unified and accessible Python library serving distortion measures. To save the time needed to install and execute the library, we make ZADU easily downloadable via the Python package index PyPI. Moreover, in line with the current trend in DR research [10, 18, 19, 29, 35], ZADU is compatible with existing Python machine learning and visualization toolboxes (e.g., `scikit-learn` [35] and `matplotlib` [14]).

ZADU differentiates from previous implementations of distortion measures from three perspectives. First, the library covers a broad range of distortion measures, with a total of 17 provided. This is over three times more than the earlier implementations with the most measures available [19]. Hence, researchers do not need to spend time searching for available codes or implementing the codes by themselves. Second, ZADU automatically optimizes the execution of multiple measures, substantially reducing the amount of computation time needed. Last, ZADU supports the computation of local pointwise distortions, which illustrates the contribution of each data point to the overall distortions. By explaining distortions in a fine-grained manner, local distortions enable a more detailed analysis of DR embeddings.

We simulate a real-world scenario of evaluating DR embeddings to assess the extent to which ZADU optimizes the execution of multiple measures. The simulation verifies that our optimization substantially reduces the total running time required for executing distortion measures. We also demonstrate using ZADU to create distortion visualizations that depict how and where the embedding suffers from distortions. We have packaged our implementation of distortion visualizations as a library called ZADUvis, enabling users to readily create the visualizations.

2 BACKGROUND AND RELATED WORK

We discuss the literature associated with distortion measures. We then review the publicly available implementations of the measures.

2.1 Distortion Measures

Distortion measures are functions that accept a high-dimensional data $\mathbf{X} = \{x_i \in \mathbb{R}^D \mid 1 \leq i \leq N\}$ and its low-dimensional embedding $\mathbf{Y} = \{y_i \in \mathbb{R}^d \mid 1 \leq i \leq N\}$ ($d < D$) as input, and then return a score that represents how well the structure of $\mathbf{Y}$ matches that of $\mathbf{X}$. The measures are either developed as a loss function of a DR technique [6, 13] or developed originally, independent of any technique [18, 25].

Distortion measures can be broadly divided into three categories—local measures, global measures, and cluster-level measures—based on their target structural granularity [18]. Local measures evaluate the extent to which the neighborhood structure of $\mathbf{X}$ is preserved in $\mathbf{Y}$. For example, Trustworthiness & Continuity (T&C) [40] and Mean relative rank error (MRRE) [26] assess the degree to which the k -nearest neighbors (k NN) of each point in $\mathbf{X}$ are no longer neighbors in the $\mathbf{Y}$, and vice versa. Neighborhood Dissimilarity [10] measures the level to which the Shared-Nearest Neighbor [8] graph structure is different in $\mathbf{X}$ and $\mathbf{Y}$. Next, cluster-level measures evaluate how well the cluster structures of $\mathbf{X}$ are preserved in $\mathbf{Y}$. The cluster is given by clustering algorithms [18] or class labels [21]. Finally, global measures evaluate the extent to which point-pairwise

Type	Measure	Ref.	provide pointwise distortions	dreval [39]	McInnes et al. [29]	Ingram et al. [15]	Jeon et al. [18]	Fujiwara et al. [10]	Espadoto et al. [9]	Colange et al. [6]	coranking [22]	pyclustering [33]	scikit-learn [35]	scipy [41]	Moor et al. [30]	Jeon et al. [19]	ZADU (Ours)
Local	Trustworthiness & Continuity	[40]	✓	△	△					○	○		△		○	○	○
	Mean Relative Rank Errors	[26]	✓												○	○	○
	Local Continuity Meta-Criteria	[4]	✓								○						○
	Neighborhood Hit	[34]	✓			○											○
	Neighbor Dissimilarity	[10]	✓					○									○
	Class-Aware Trustworthiness & Continuity	[6]	✓							○							○
Cluster-level	Procrustes Measure	[12]	✓														○
	Steadiness & Cohesiveness	[18]	✓				○	○									○
	Distance Consistency	[37]	✓														○
	Internal Clustering Validation Measures	[21]	✓									○	○				○
Global	Clustering + External Clustering Validation Measures	[42]	✓									○	○				○
	Stress	[23, 24]	✓						○						○	○	○
	Kullback-Leibler Divergence	[13]	✓												○	○	○
	Distance-to-Measure	[3]	✓													○	○
	Topographic Product	[1]	✓														○
	Pearson's correlation coefficient r	[11]	✓											○			○
	Spearman's rank correlation coefficient ρ	[36]	✓											○			○

Table 1: The overview of the distortion measures provided by ZADU (row) and their publicly available implementations (column). The publicized implementations that fully implement the corresponding measures are highlighted in red background and circle. The ones that implement only half of the pair of measures are highlighted in light red background and triangle.

distances remain consistent. For instance, Pearson's correlation coefficient r quantifies how the ranking of point pairs based on their distances varies between $\mathbf{X}$ and $\mathbf{Y}$.

As diverse DR techniques emphasize different facets of data, employing multiple distortion metrics with varying granularity levels is crucial for the comprehensive evaluation of DR embeddings. Therefore, while designing ZADU, we try not only to maximize the number of supported distortion measures but also to have an even distribution of all types of measures (Table 1, Section 3.2).

2.2 Implementations of Distortion Measures

Despite the importance of reliability evaluations when utilizing DR, there is a lack of a unified implementation that provides distortion measures. The majority of implementations is in publicly accessible repositories contributed by the studies on DR [5, 10, 19, 29, 30]. However, each implementation has a limited number of supported distortion measures (Table 1). Moreover, installing, compiling, and executing from such scattered code is time-consuming.

An alternative way is to use the distortion measures provided by popular machine learning libraries (e.g., `scikit-learn` [35]). These libraries are easy to install and execute, and also likely to be highly optimized. However, as general-purpose machine learning toolboxes, they offer limited support for distortion measures (Table 1). We aim to develop a library that (1) is easily downloadable and executable, similar to the widely-used machine learning libraries, while (2) supporting a broader range of distortion measures.

3 ZADU

We first present the supported measures and the interface of ZADU. We then delve into the functionalities offered by the library that facilitate the efficient and reliable analysis of DR embeddings.

3.1 Supported Distortion Measures

The list of distortion measures to be included in the library is determined through a literature review on DR and their evaluation (Section 2). Different distortion measures evaluate the preservation of the data structure at varying levels of granularity (e.g., neighborhood, cluster, and global structure; Section 2.1). The simultaneous use of multiple measures having different granularity is essential for comprehensively evaluating DR embeddings [9, 19, 30]. Thus, we try to maximize both the number of supported measures and

the diversity in terms of the structural granularity that the measures focus on. As a result, we select seven local measures, four cluster-level measures, and six global measures (Table 1). Please refer to Appendix A for the detailed procedure for computing each measure.

3.2 Interface

ZADU provides two different interfaces for executing distortion measures. The first is to use the main class that is named after our library (i.e., `zadu`). In designing the main interface, our focus is on reusing both the code and the computing resources so that users can save time. With regard to reusing code, we force users to write a specification that defines the measures to be executed ("id" in Code 1) along with their hyperparameters ("params"). By reusing the specifications, users can perform an identical evaluation on multiple datasets. This is commonly done in practice to enhance the generalizability of the evaluation [9, 19, 30]. As for reusing the computing results, we require users to register the original high-dimensional dataset (`hd`) once, along with its specifications. This dataset can then be reused repeatedly. This is because the evaluation of DR is usually done by comparing multiple embeddings of a single high-dimensional dataset. The distortion measures can then be executed by invoking measure method while giving the embedding (`ld`) as an argument, which returns the scores from the distortion measures.

```

1 from zadu import zadu
2
3 hd, ld = load_datasets()
4 spec = [{
5     "id": "tnc",
6     "params": { "k": 20 },
7 }, {
8     "id": "snc",
9     "params": { "k": 30, "clustering": "hdbscan" }
10 }]
11
12 scores = zadu.ZADU(spec, hd).measure(ld)
13 print("T&C:", scores[0])
14 print("S&C:", scores[1])

```

Code 1: Using the main class of ZADU to compute the Trustworthiness & Continuity (tnc) and Steadiness & Cohesiveness (snc) scores of a given embedding (`ld`) based on its original data (`hd`).

An alternative interface is to directly invoke the functions that define each distortion measure (Code 2). However, executing multiple measures in this way does not take advantage of optimization (Section 3.3.1). Hence, more computation time is needed compared to using the main class (Code 1).

```
1 from zadu.measures import *
2
3 mrre = mean_relative_rank_error.measure(hd, ld)
4 pr = pearson_r.measure(hd, ld)
```

Code 2: Accessing the internal functions of ZADU to execute Mean Relative Rank Errors and Pearson's correlation coefficient r .

3.3 Functionalities

We outline the functionalities of ZADU that enable the effective evaluation and analysis of DR embeddings.

3.3.1 Optimizing the Execution of Multiple Measures

Utilizing multiple distortion measures simultaneously is common in practice [19, 30]. For example, Espadoto et al. [9] proposed to aggregate multiple measures by averaging them. However, using more measures leads to increased computational demands.

To reduce the computation time running multiple distortion measures, ZADU automatically optimizes the execution of the measures. The primary goal of the optimization is to minimize the computational overhead associated with three key preprocessing blocks: pairwise distance computation, pointwise distance ranking computation, and k NN identification. The pairwise distance computation is done by constructing a distance matrix in both the original and the embedded spaces utilizing a specified distance function (e.g., Euclidean distance or cosine similarity). During the pointwise distance ranking computation stage, the ranking of all data points with respect to each individual data point x is set based on their distance from x . This is also done in both the original and the embedded spaces. Lastly, k NN identification involves locating the top- k closest data points of each point in the original and embedded spaces.

The optimization works as follows. Given a specification (refer to Section 3.2), ZADU extracts a list of requisite preprocessing units. The library then establishes an execution order for the blocks while maximizing the reuse of computed results. For instance, if both the distance matrix and the k NN index are needed, the outcome of the former computation is reused to compute the latter. Similarly, if the specifications require the computation of both k_1 NN and k_2 NN, where $k_1 > k_2$, the k_2 NN can be acquired by slicing the k_1 NN. Once the execution order and dependencies are ascertained, ZADU runs preprocessing. The preprocessing results are stored in the RAM and subsequently injected into each function that defines a distortion measure to derive the final scores.

The effectiveness of our optimization increases as more distortion measures are executed simultaneously. We validate that the optimization substantially reduces the execution time of distortion measures through our quantitative evaluation (Section 4).

3.3.2 Computing Pointwise Local Distortions

ZADU enables users to obtain local pointwise distortions, which indicate how each point contributes to the overall distortions. Such functionality improves the usability of our library as local distortions help users in performing enhanced analysis of DR embeddings. For example, we can aggregate local distortions in class labels to reveal which class is vulnerable to the distortions. Moreover, we can visualize local distortions [18, 27], which facilitates a more accurate analysis of the original high-dimensional data [18]. We discuss this application in more detail in Section 4.2.

We can obtain local pointwise distortions by raising the `return_local` flag. When the flag is raised, the library returns the local distortions along with the aggregated scores (See Code 3).

```
1 from zadu import zadu
2
3 spec = [{
4     "id" : "dtm",
5     "params": {}
6 }, {
7     "id" : "mrre",
8     "params": { "k": 30 }
9 }]
10
11 zadu_obj = zadu.ZADU(spec, hd, return_local=True)
12 global_, local_ = zadu_obj.measure(ld)
13 print("MRRE local distortions:", local_[1])
```

Code 3: Obtaining local pointwise distortion from ZADU by raising the `return_local` flag. If a specified distortion measure produces local pointwise distortion as an intermediate result, it returns a list of pointwise distortions when the flag is raised.

The computation of pointwise local distortions is available only for some local measures and cluster-level measures (See “provide pointwise distortions” column in Table 1). For example, T&C and MRREs produce final scores as an average of local distortions. Steadiness & Cohesiveness [18] computes pointwise distortion by aggregating partial cluster-level distortions. When the flag is raised, ZADU returns a list consisting of local pointwise distortions for the available measures; it otherwise returns None.

3.4 Implementation

ZADU is a Python library that can be installed via PyPI with just a single command. Scalability is a key consideration in implementing ZADU. We maximize the utilization of matrix computation and incorporate highly optimized open-source libraries for computationally heavy tasks (e.g., `faiss` [20] for k NN identification). To simplify the installation and execution, the library runs only on CPUs.

While implementing the measures, we reuse the previous open-source implementations if available. For example, for T&C, MRRE, Stress, DTM, and KL divergence, we adopt the code provided by Jeon et al. [19] (the second last column of Table 1). For Steadiness & Cohesiveness, we use the code shared by the authors. We still revise these codes to fit our optimization pipeline (Section 3.3.1), to make them return local pointwise distortions (Section 3.3.2), and to eliminate GPU dependencies. The remaining measures are carefully implemented by referring to the papers in which they were first introduced. The source code is available at github.com/hj-n/zadu.

4 RUNTIME ANALYSIS

4.1 Objectives and Design

We test whether our optimization pipeline (Section 3.3.1) reduces the time needed to evaluate DR embeddings. We simulate a scenario in which we try to optimize the hyperparameters of a DR technique using multiple distortion measures that have common preprocessing blocks. We evaluate the running time for optimization with and without the optimization. We use datasets with diverse characteristics, e.g., the number of points and dimensionality. We compare how the running time of evaluation differs on average as we switch on the optimization.

Optimization For a given dataset, we measure the time required to run Bayesian optimization [38] for finding the optimal value of two hyperparameters in UMAP [29]: `nearest_neighbors` and `minimum_distance` [29]. The search range of two hyperparameters is set as (2, 200) and (0.01, 0.99), respectively, following the recommendation of the official documentation¹. For Bayesian optimization, we use the Python implementation of Nogueira [31] with the default hyperparameter setting.

¹umap-learn.readthedocs.io

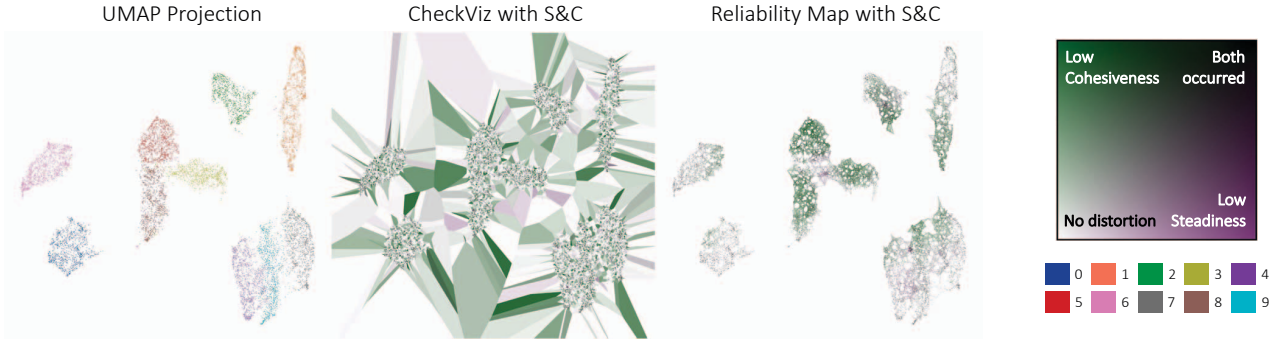


Figure 1: The UMAP embedding of the MNIST dataset (leftmost column), and two distortion visualizations generated by ZADUVis: CheckViz [27] and the Reliability Map [18]. The distortion visualizations depict how each region of the given embedding suffers from the distortions that lower the Steadiness & Cohesiveness (S&C) scores. The visualizations follow the 2D colormap proposed by Lespinats and Aupetit [27] (rightmost column). Combined with ZADU, ZADUVis helps practitioners easily generate distortion visualizations on a matplotlib canvas.

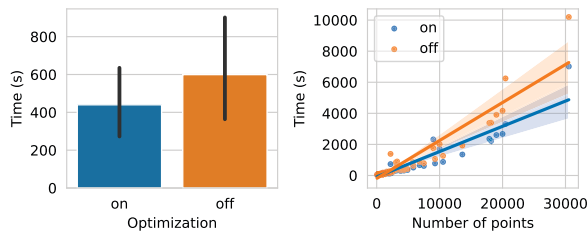


Figure 2: The results of the runtime analysis (Section 4). (left) ZADU's optimization substantially reduces the runtime for the optimization of DR embeddings. (right) The extent to which the optimization reduces the runtime increases as the size of the datasets increases. The shaded area in the right figure depicts the 95% confidence interval.

Distortion measures For the distortion measures, we use T&C, MRRE, Steadiness & Cohesiveness, Distance-to-Measure, and Kullback-Leibler divergence. All the measures share pairwise distance matrix computation as a common preprocessing block. The first three measures also share k NN identification. As a loss function, we use an average of five measures, following Espadoto et al. [9].

Datasets We apply the optimization to the 96 publicly available high-dimensional datasets gathered by a previous study [17]. Every dataset is standardized before applying the optimization process.

4.1.1 Results

Fig. 2 depicts the result. ZADU is 1.5 times faster with optimization than without it on average, verifying the effectiveness of the optimization pipeline. We also discover that the difference in runtime between with and without optimization increases as the number of points in the dataset increases (as indicated by the steeper orange regression line in Fig. 2b compared to the blue regression line). This finding further supports the scalability benefits of ZADU. Overall, our results demonstrate that ZADU substantially reduces the time required for practitioners to evaluate DR embeddings.

4.2 Application: Visualizing Local Distortions

Various distortion visualization methods [18, 27] have been proposed to provide insights into the extent to which each region is affected by distortions. CheckViz [27] (Fig. 1 second column), for example, decomposes the scatterplot that represents a DR embedding using a Voronoi diagram, and then encodes the distortion of each point as a color of the corresponding Voronoi cell. Reliability Map [18] (Fig. 1 third column) constructs a k NN graph in the embedded space and encodes the distortions of each point on the incident graph edges.

We present the implementation of local distortion visualizations as an application of ZADU. We develop ZADUVis, a Python library

that provides CheckViz and the Reliability Map as representative distortion visualizations. ZADUVis takes local pointwise distortions generated by ZADU as input and uses them to generate distortion visualizations. Integrated with matplotlib [14], ZADUVis allows users to render a distortion visualization without time-consuming extra implementation (Code 4). Extending our application to a more complex visual analytics system would be an interesting direction.

```
1 from zadu import zadu
2 from zaduvis import zaduvis
3 import matplotlib.pyplot as plt
4 from sklearn.manifold import TSNE
5
6 ## load datasets and generate an embedding
7 hd = load_mnist()
8 ld = TSNE().fit_transform(hd)
9
10 ## Computing local pointwise distortions
11 spec = [{"id": "snc", "params": {"k": 50}}]
12 zadu_obj = zadu.ZADU(spec, hd, return_local=True)
13 global_, local_ = zadu_obj.measure(ld)
14 l_s = local_[0]["local_steadiness"]
15 l_c = local_[0]["local_cohesiveness"]
16
17 ## Visualizing local distortions
18 fig, ax = plt.subplots(1, 2, figsize=(20, 10))
19 zaduvis.checkviz(ld, l_s, l_c, ax=ax[0])
20 zaduvis.reliability_map(ld, l_s, l_c, ax=ax[1])
```

Code 4: Visualizing CheckViz [27] and the Reliability Map [18] using ZADUVis and matplotlib. ZADUVis gets the embedding and the local distortions made by ZADU as arguments and generates distortion visualization. The rendered image of this code is depicted in the second (CheckViz) and third (Reliability Map) columns of Fig. 1.

5 CONCLUSION

Utilizing distortion measures has so far been time-consuming due to the lack of a well-established implementation. To address this issue, we present ZADU, a Python library that allows easy and scalable execution of distortion measures. We believe that ZADU will mitigate the challenges associated with the evaluation of DR embeddings, promoting the design and development of visual analytics applications for high-dimensional data.

We plan to extend our library into JavaScript, making it compatible with a wider range of existing visualizations [2] and DR [7] toolboxes. Investigating how each distortion measure operates in more detail will also be an interesting direction. We would also like to provide guidelines for utilizing distortion measures.

ACKNOWLEDGMENTS

This work was supported by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (No. 2023R1A2C200520911).

REFERENCES

- [1] H.-U. Bauer and K. R. Pawelzik. Quantifying the neighborhood preservation of self-organizing feature maps. *IEEE Transactions on neural networks*, 3(4):570–579, 1992.
- [2] M. Bostock, V. Ogievetsky, and J. Heer. D³: Data-driven documents. *IEEE Trans. on Visualization and Computer Graphics*, 17(12):2301–2309, 2011. doi: 10.1109/TVCG.2011.185
- [3] F. Chazal, D. Cohen-Steiner, and Q. Mérigot. Geometric inference for probability measures. *Foundations of Computational Mathematics*, 11(6):733–751, 2011.
- [4] L. Chen and A. Buja. Local multidimensional scaling for nonlinear dimension reduction, graph drawing, and proximity analysis. *Journal of the American Statistical Association*, 104(485):209–219, 2009. doi: 10.1198/jasa.2009.0111
- [5] A. Cockburn, A. Karlson, and B. B. Bederson. A review of overview+detail, zooming, and focus+context interfaces. *ACM Computing Surveys*, 41(1), Jan. 2009. doi: 10.1145/1456650.1456652
- [6] B. Colange, J. Peltonen, M. Aupetit, D. Dutykh, and S. Lespinats. Steering distortions to preserve classes and neighbors in supervised dimensionality reduction. In *Advances in Neural Information Processing Systems*, vol. 33, pp. 13214–13225, 2020.
- [7] R. Cutura, C. Kralj, and M. Sedlmair. Druidjs — a javascript library for dimensionality reduction. In *2020 IEEE Visualization Conference (VIS)*, pp. 111–115, 2020. doi: 10.1109/VIS47514.2020.00029
- [8] L. Ertöz, M. Steinbach, and V. Kumar. A new shared nearest neighbor clustering algorithm and its applications. In *Workshop on Clustering High Dimensional Data and its Applications at 2nd SIAM Int. Conf. on Data mining*, pp. 105–115, 2002.
- [9] M. Espadoto, R. M. Martins, A. Kerren, N. S. T. Hirata, and A. C. Telea. Toward a quantitative survey of dimension reduction techniques. *IEEE Transactions on Visualization and Computer Graphics*, 27(3):2153–2173, 2021. doi: 10.1109/TVCG.2019.2944182
- [10] T. Fujiwara, Y.-H. Kuo, A. Ynnerman, and K.-L. Ma. Feature learning for nonlinear dimensionality reduction toward maximal extraction of hidden patterns. In *2023 IEEE 16th Pacific Visualization Symposium (PacificVis)*, pp. 122–131, 2023. doi: 10.1109/PacificVis56936.2023.00021
- [11] X. Geng, D.-C. Zhan, and Z.-H. Zhou. Supervised nonlinear dimensionality reduction for visualization and classification. *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)*, 35(6):1098–1107, 2005.
- [12] Y. Goldberg and Y. Ritov. Local procrustes for manifold embedding: a measure of embedding quality and embedding algorithms. *Machine learning*, 77:1–25, 2009.
- [13] G. Hinton and S. Roweis. Stochastic neighbor embedding. In *Proc. of the 15th Int. Conf. on Neural Information Processing Systems*, p. 857–864. MIT Press, Cambridge, MA, USA, 2002.
- [14] J. D. Hunter. Matplotlib: A 2d graphics environment. *Computing in science & engineering*, 9(03):90–95, 2007.
- [15] S. Ingram and T. Munzner. Dimensionality reduction for documents with nearest neighbor queries. *Neurocomputing*, 150:557–569, 2015. doi: 10.1016/j.neucom.2014.07.073
- [16] H. Jeon, M. Aupetit, S. Lee, H.-K. Ko, Y. Kim, and J. Seo. Distortion-aware brushing for interactive cluster analysis in multidimensional projections, 2022. doi: 10.48550/ARXIV.2201.06379
- [17] H. Jeon, M. Aupetit, D. Shin, A. Cho, S. Park, and J. Seo. Sanity check for external clustering validation benchmarks using internal validation measures, 2022. doi: 10.48550/ARXIV.2209.10042
- [18] H. Jeon, H.-K. Ko, J. Jo, Y. Kim, and J. Seo. Measuring and explaining the inter-cluster reliability of multidimensional projections. *IEEE Trans. on Visualization and Computer Graphics*, 28(1):551–561, 2021. doi: 10.1109/TVCG.2021.3114833
- [19] H. Jeon, H.-K. Ko, S. Lee, J. Jo, and J. Seo. Uniform manifold approximation with two-phase optimization. In *2022 IEEE Visualization and Visual Analytics (VIS)*, pp. 80–84, 2022. doi: 10.1109/VIS54862.2022.00025
- [20] J. Johnson, M. Douze, and H. Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2019.
- [21] P. Joia, D. Coimbra, J. A. Cuminato, F. V. Paulovich, and L. G. Nonato. Local affine multidimensional projection. *IEEE Trans. Vis. Comput. Graphics*, 17(12):2563–2571, 2011. doi: 10.1109/TVCG.2011.220
- [22] G. Kraemer, M. Reichstein, and M. D. Mahecha. dimRed and coRanking—Unifying Dimensionality Reduction in R. *The R Journal*, 10(1):342–358, 2018.
- [23] J. Kruskal. Multidimensional scaling by optimizing goodness of fit to a nonmetric hypothesis. *Psychometrika*, 29:1–27, 1964. doi: 10.1007/BF02289565
- [24] J. B. Kruskal. Nonmetric multidimensional scaling: a numerical method. *Psychometrika*, 29(2):115–129, 1964.
- [25] J. A. Lee and M. Verleysen. *Nonlinear Dimensionality Reduction*. Springer-Verlag New York, 2007. doi: 10.1007/978-0-387-39351-3
- [26] J. A. Lee and M. Verleysen. Quality assessment of dimensionality reduction: Rank-based criteria. *Neurocomputing*, 72(7):1431–1443, 2009. doi: 10.1016/j.neucom.2008.12.017
- [27] S. Lespinats and M. Aupetit. Checkviz: Sanity check and topological clues for linear and non-linear mappings. *Computer Graphics Forum*, 30(1):113–125, 2011. doi: 10.1111/j.1467-8659.2010.01835.x
- [28] R. M. Martins, D. B. Coimbra, R. Minghim, and A. Telea. Visual analysis of dimensionality reduction quality for parameterized projections. *Computers & Graphics*, 41:26–42, 2014.
- [29] L. McInnes, J. Healy, and J. Melville. Umap: Uniform manifold approximation and projection for dimension reduction, 2020.
- [30] M. Moor, M. Horn, B. Rieck, and K. Borgwardt. Topological autoencoders. In H. D. III and A. Singh, eds., *Proceedings of the 37th International Conference on Machine Learning*, vol. 119, pp. 7045–7054. PMLR, 13–18 Jul 2020.
- [31] F. Nogueira. Bayesian Optimization: Open source constrained global optimization tool for Python, 2014.
- [32] L. G. Nonato and M. Aupetit. Multidimensional projection for visual analytics: Linking techniques with distortions, tasks, and layout enrichment. *IEEE Trans. on Visualization and Computer Graphics*, 25(8):2650–2673, 2019. doi: 10.1109/TVCG.2018.2846735
- [33] A. V. Novikov. Pylustering: Data mining library. *Journal of Open Source Software*, 4(36):1230, 2019.
- [34] F. V. Paulovich, L. G. Nonato, R. Minghim, and H. Levkowitz. Least square projection: A fast high-precision multidimensional projection technique and its application to document mapping. *IEEE Transactions on Visualization and Computer Graphics*, 14(3):564–575, 2008. doi: 10.1109/TVCG.2007.70443
- [35] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, et al. Scikit-learn: Machine learning in python. *the Journal of machine Learning research*, 12:2825–2830, 2011.
- [36] S. Sidney. Nonparametric statistics for the behavioral sciences. *The Journal of Nervous and Mental Disease*, 125(3):497, 1957.
- [37] M. Sips, B. Neubert, J. P. Lewis, and P. Hanrahan. Selecting good views of high-dimensional data using class consistency. *Computer Graphics Forum*, 28(3):831–838, 2009. doi: 10.1111/j.1467-8659.2009.01467.x
- [38] J. Snoek, H. Larochelle, and R. P. Adams. Practical bayesian optimization of machine learning algorithms. *Advances in neural information processing systems*, 25, 2012.
- [39] C. Soneson. *dreval: Evaluate Reduced Dimension Representations*, 2022. R package version 0.1.5.
- [40] J. Venna and S. Kaski. Local multidimensional scaling. *Neural Networks*, 19(6):889–899, 2006. doi: 10.1016/j.neunet.2006.05.014
- [41] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright, et al. Scipy 1.0: fundamental algorithms for scientific computing in python. *Nature methods*, 17(3):261–272, 2020.
- [42] R. Xiang, W. Wang, L. Yang, S. Wang, C. Xu, and X. Chen. A comparison for dimensionality reduction methods of single-cell rna-seq data. *Frontiers in Genetics*, 12, 2021.